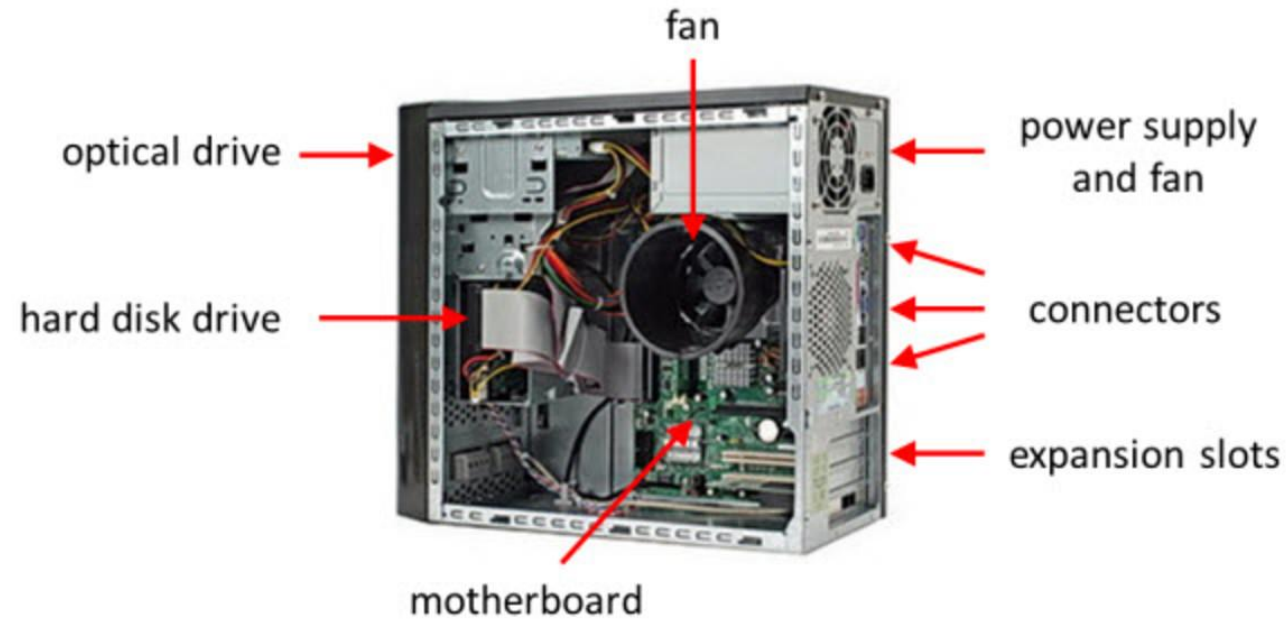


Topics to be covered

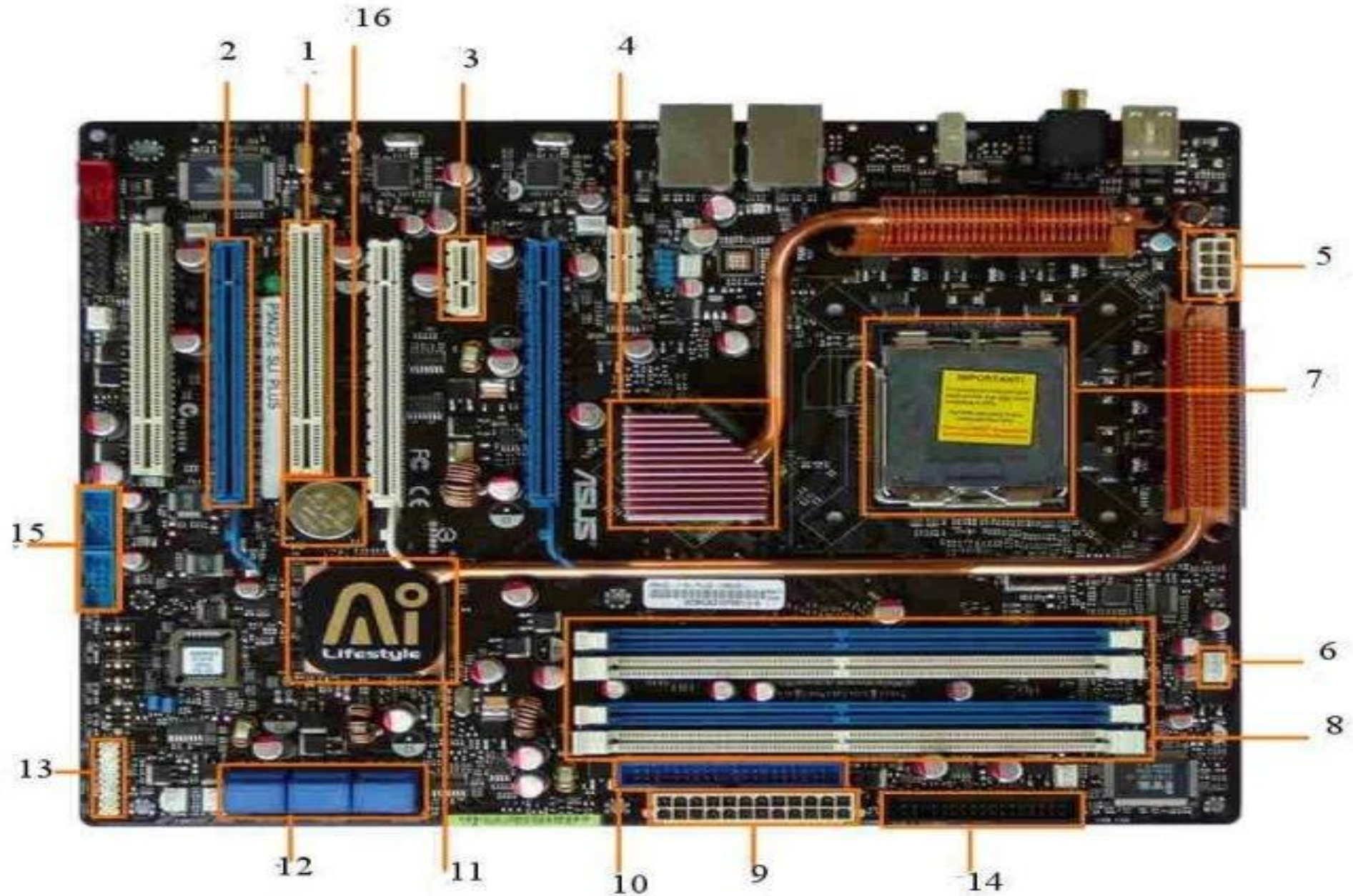
- **CPU Architecture**
 - Processor organization, register organization, instruction cycle
- **Introduction to Buses**
 - Interfacing buses (circuit diagrams not necessary), concepts of address bus, data bus and control bus, bus width (circuit diagrams not necessary)
- **Input and Output**
 - External devices, I/O modules, Why DMA?, DMA

Internal parts of CPU



- Alternatively known as a **bus slot** or **expansion port**, an **expansion slot** is a connection or port inside a computer on the motherboard. It provides an installation point for a hardware expansion card to be connected. For example, if you wanted to install a new video card in the computer, you'd purchase a video expansion card and install that card into the compatible expansion slot. It provides additional features to a computer such as video, sound, advanced graphics, Ethernet or memory.

various slots on motherboard



various slots on motherboard

- A typical Asus P5N32-E SLI motherboard with constituent components is given below:
- **1. PCI SLOT (Peripheral Component Interconnect)** : PCI slots are used to Insert or install Add-on cards, such as LAN cards, Sound cards, Capture cards and TV tuner cards. There are usually anywhere from 1 to 6 PCI slots available on the motherboard(above board has 2 PCI slots.), they have decreased in number and are being replaced by the PCI Express 1x slots.
- **2. PCI-E 16x Slot** : the most common slot for Graphics cards, the PCI Express 16x slots provides 16 separate lanes or data transfer. These are the 16x speed versions, which are currently the fastest. PCI-E16x allows up to 4 GB/s of peak bandwidth per direction, and up to 8 GB/s concurrent bandwidth.
- **3. PCI-E 1x Slot** : Single slot - In the PCIe 1x generation, each lane (1x) carries 250 MB/s compared to 133 MB/s for the PCI slots. These can be used for expansion cards such as Sound Cards, or Ethernet Cards.

various slots on motherboard

- **4. Northbridge:** This allows communication between the CPU and the system memory and PCI-E slots. It is a focal Point of Motherboard and It is also called as Memory Controller Hub.
- **5. ATX 12V 2X and 4 Pin Power Connection :** This is one of two power connections that supply power to the motherboard. This is the 4-pin connector which **supplies electrical current to your CPU**. This connection will come from your Power Supply.
- **6. CPU-Fan Connection:** This is where the CPU fan will connect. Using this connection over one of the power supply will allow the motherboard to control the speed of the fan, based on the CPU temperature.
- **7. Socket:** This is where the CPU will plug in.
- **8. DIMM slots:** DIMM (dual in-line memory module) slots are **the place on your motherboard where the RAM goes**. DIMM's are by far and away the most used memory types in today's computers. They vary in speeds and standards however and they need to match up to what your motherboard has been designed to take. The four standards of DIMM's being used at the moment are SDR (Single Data Rate), DDR (Double Data Rate), DDR2 and DDR3. The speeds of memory can vary between 66Mhz to 1600Mhz.

various slots on motherboard

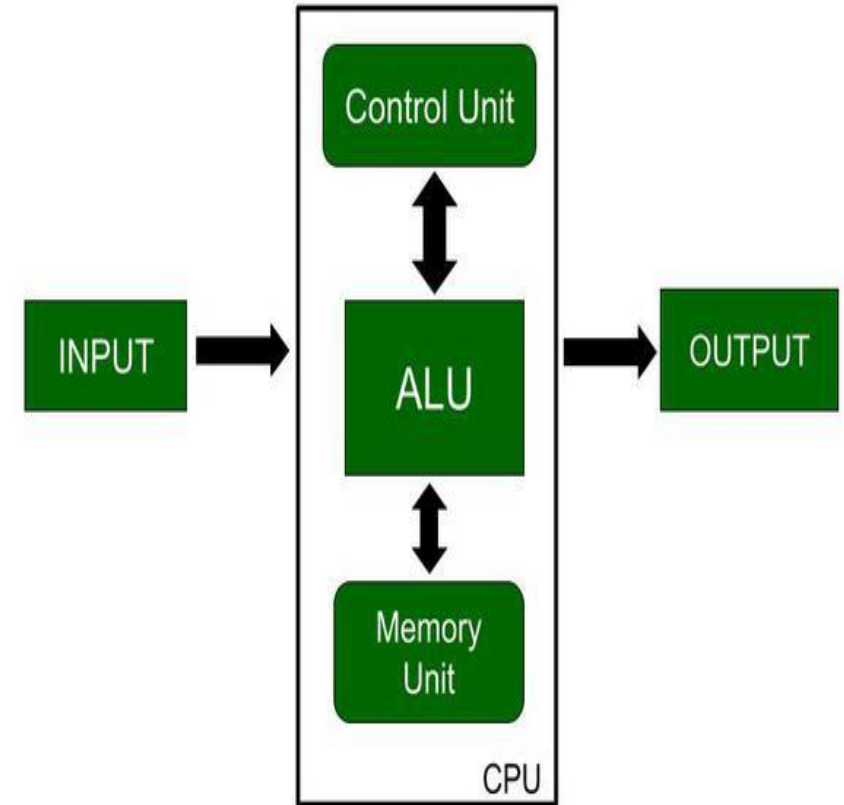
- **9. ATX Power Connector:** This is the second of two power connections. This is the main power connection for the motherboard, and comes from the Power Supply.
- **10. IDE connectors or PATA connectors :** IDE full form is Integrated Device Electronics. it supports IDE devices, such as Hard disks and CD and DVD drives. Most drives today come with SATA connections.
- **11. Southbridge:** This is the controller for components such as the PCI slots, onboard audio, and USB connections.
- **12. SATA Connections :** SATA full form is Serial Advanced Technology Attachment. These are connect with serial ATA devices, such as Hard disk drives and CD or DVD drives.

various slots on motherboard

- **13. Front Panel Connections:** this is where we will hook in the connections from the case. These are mostly the different lights on the case, such as power on, hard drive activity etc. It is **a block of connectors on a motherboard that control the power on, power reset, beep code speaker and the LED light indicators on your PC case/chassis.**
- **14. FDD Connection:** The FDD is the Floppy Disk controller. Floppy Drive Connector is used to connect floppy drives. It supports two floppy drives.
- **15. External USB Connections:** There are usually a couple of these ports located on each motherboard used for connecting pen drives and external hard drives, like Ipods or Mp3 players.
- **16. CMOS battery :** The CMOS (complementary metal oxide semiconductor) battery in the computer is **used to store the hardware configuration settings.** It also keeps a record of the date and time when the computer is switched off. The CMOS battery is inside the computer and attached to the system board (motherboard).

Processor organization

- CPU is the brain of the computer.
- CPU consists of 3 major units, which are:
- Memory or Storage Unit
- Control Unit
- ALU(Arithmetic Logic Unit)



Overview of CPU design

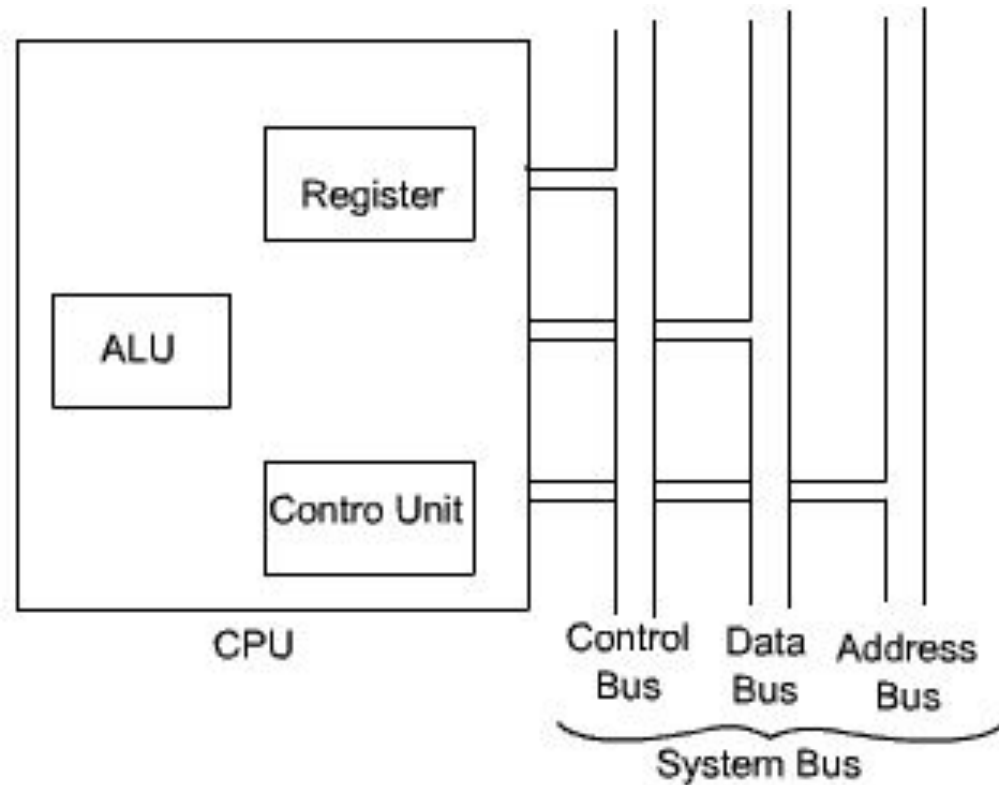
- The operation or task that must perform by CPU.
- Fetch Instruction: The CPU reads an instruction from memory.
- Interpret Instruction: The instruction is decoded to determine what action is required
- Fetch Data: The execution of an instruction may require reading data from memory or I/O module.
- Process data: The execution of an instruction may require performing some arithmetic or logical operation on data.
- Write data: The result of an execution may require writing data to memory or an I/O module.
- To do these tasks, it should be clear that the CPU needs to store some data temporarily. It must remember the location of the last instruction so that it can know where to get the next instruction. It needs to store instructions and data temporarily while an instruction is being executed. In other words, the CPU needs a small internal memory. These storage location are generally referred as registers.

Overview of CPU design

- The major components of the CPU are an arithmetic and logic unit (ALU) and a control unit (CU).
- The ALU does the actual computation or processing of data.
- The CU controls the movement of data and instruction into and out of the CPU and controls the operation of the ALU.
- The CPU is connected to the rest of the system through system bus. Through system bus, data or information gets transferred between the CPU and the other component of the system.
- The system bus may have three components:

Overview of CPU design

- The internal organization of CPU



Overview of CPU design

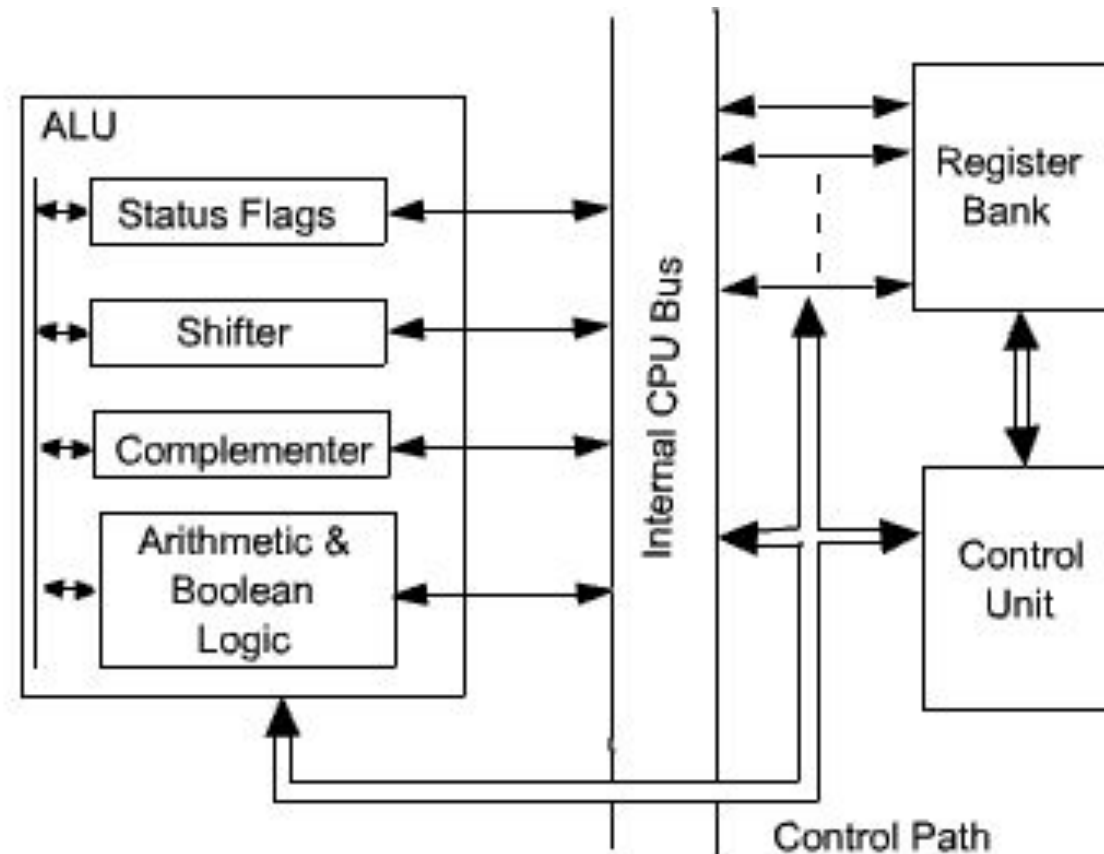
- Data Bus: Data bus is used to transfer the data between main memory and CPU.
- Address Bus: Address bus is used to access a particular memory location by putting the address of the memory location.
- Control Bus: Control bus is used to provide the different control signal generated by CPU to different part of the system. As for example, memory read is a signal generated by CPU to indicate that a memory read operation has to be performed. Through control bus this signal is transferred to memory module to indicate the required operation.

Overview of CPU design

- There are three basic components of CPU:
- register bank, ALU and Control Unit.
- There are several data movements between these units and for that an internal CPU bus is used.
- Internal CPU bus is needed to transfer data between the various registers and the ALU.

Overview of CPU design

- CPU with the system Bus



Overview of CPU design

- **Status flags** - reflect the results of computations (add, subtract, multiply, divided) executed by the processor
- **shifter:-** Shift Registers are used **for data storage or for the movement of data** and are therefore commonly used inside calculators or computers to store data such as two binary numbers before they are added together. As the name implies, a shifter **shifts a binary number left or right by a specified number of positions**. Several kinds of commonly used shifters exist: Logical shifter—shifts the number to the left or right and fills empty spots with 0's.
- ALU shift operations cause operand to shift left or right (depending on the opcode).
- In assembly language mnemonic form an opcode is a command such as MOV or ADD or JMP. For example. MOV AL, 34h.
- **complementer:-** complement (invert) the logic value of individual bits of the data word
- **Arithmetic and Boolean logic:** perform arithmetic and Boolean functions.
- A register is a small amount of storage available as part of a CPU. The control unit tells the ALU what operation to perform on that data, and the ALU **stores the result in an output register**.

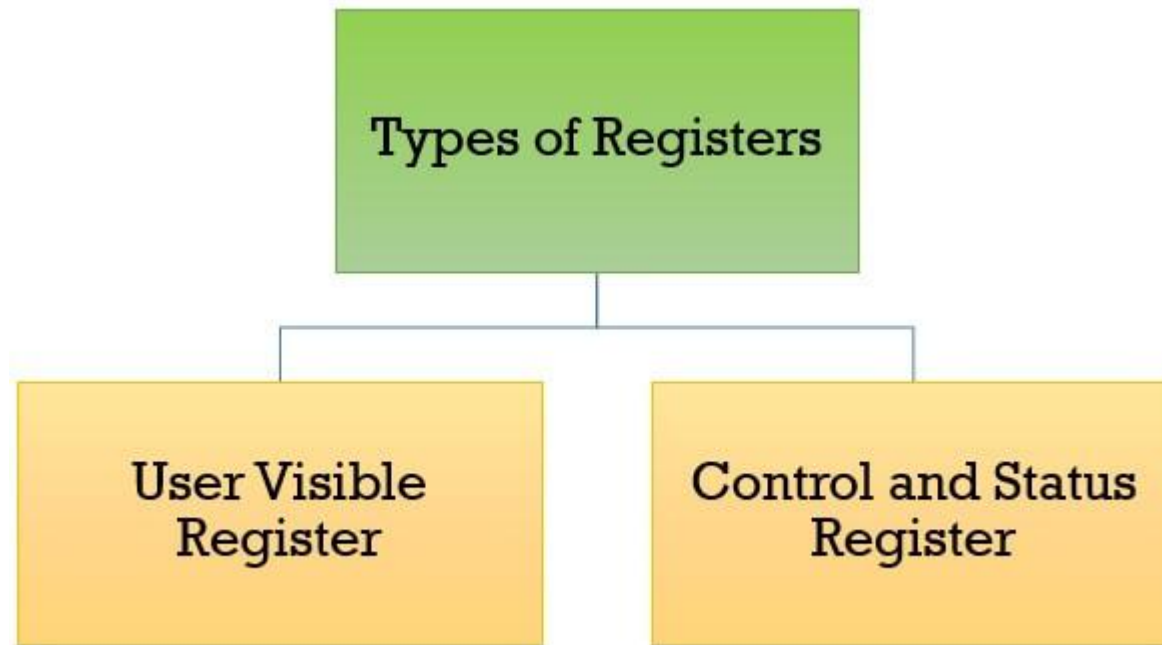
Register Organization

- A computer system employs a memory hierarchy. At the highest level of hierarchy, memory is faster, smaller and more expensive. Within the CPU, there is a set of registers which can be treated as a memory in the highest level of hierarchy.
- **Register organization** is the arrangement of the registers in the processor. The processor designers decide the organization of the registers in a processor. Different processors may have different register organization. The registers in the CPU can be categorized into two groups:

What is Register?

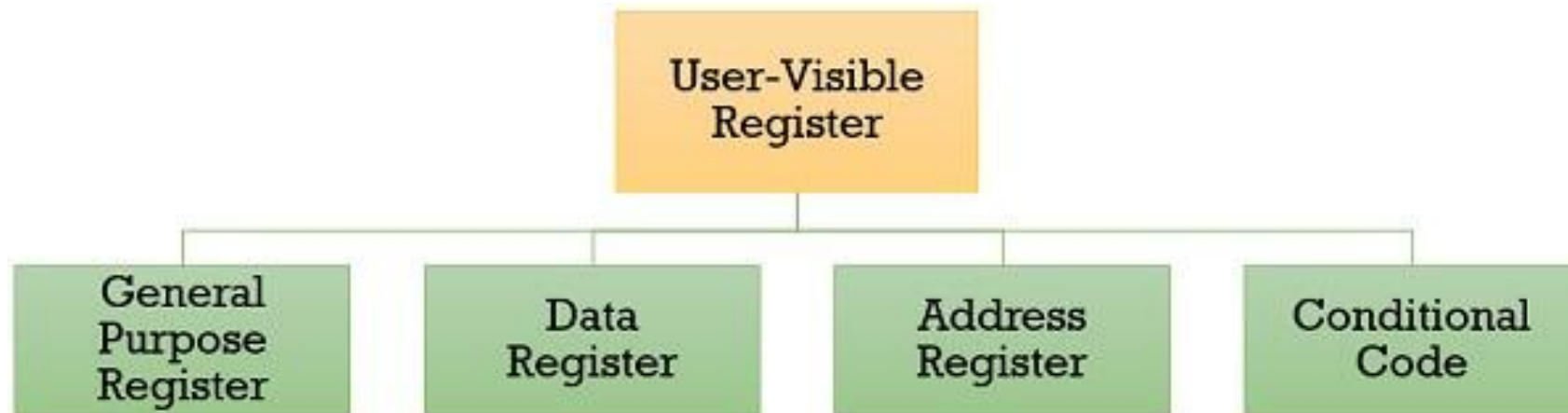
- **Registers** are the *smaller* and the *fastest* accessible **memory units** in the central processing unit (CPU). According to memory hierarchy, the registers in the processor, function a **level above** the **main memory** and **cache memory**. The registers used by the central unit are also called as **processor registers**.
- A register can hold the *instruction, address location, or operands*.

Types of Registers



User-visible registers

- These registers are visible to the assembly or machine language programmers and they use them effectively to **minimize the memory references** in the instructions. Well, these registers can only be **referenced using the machine or assembly language**.
- The user-visible registers can be categorized as follows:



User-visible registers

- The **general-purpose registers** keep both the **addresses** or the **data**. Although we have separate **data registers** and **address registers**. The general purpose register also accepts the **intermediate results** in the course of program execution.
- Well, the programmers can **restrict** some of the general-purpose registers to **specific functions**. Like, some registers are specifically used for stack operations or for floating-point operations.
- **Data registers** may be used to hold only data and cannot be employed in the calculation of an operand address.
- **Address registers** Now, the address registers contain the **address of an operand** or it can also act as a **general-purpose register**. An address register may be dedicated to a certain **addressing mode**. Examples include the following:

User-visible registers

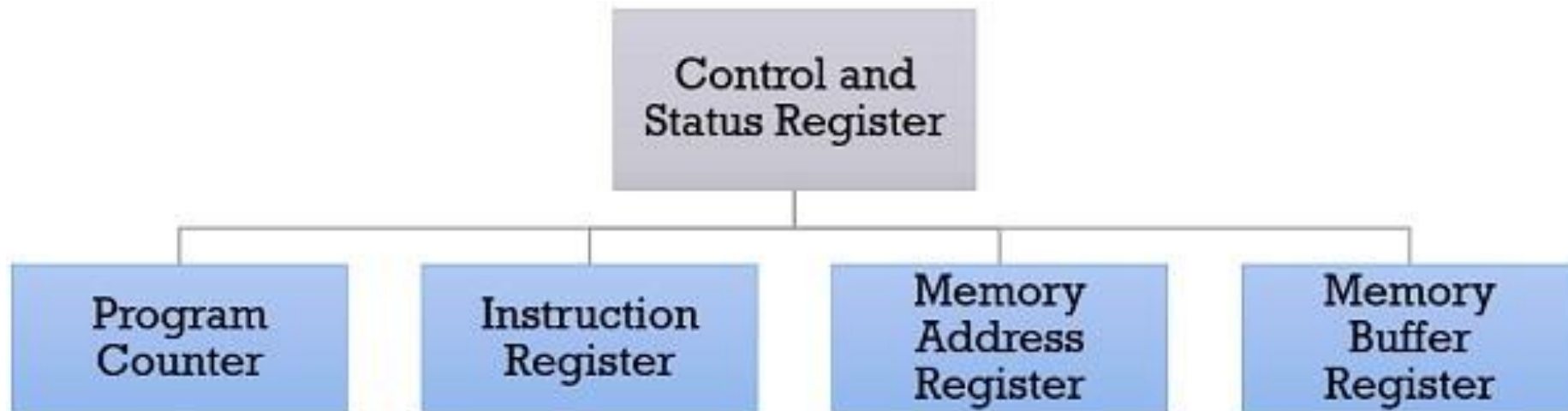
- **Segment pointer register** :A memory divided in segments, requires a segment register to **hold the base address of the segment**. There can be multiple segment registers. As one segment register can be employed to hold the base address of the segment occupied by the operating system. The other segment register can hold the base address of the segment allotted to the processor.
- **[Note: Base address - An address that serves as a reference point for other addresses.** For example, a base address could indicate the beginning of a program. The address of every instruction in the program could then be specified by adding an offset to the base address.]
- Segment registers are basically *memory pointers* located inside the CPU.
- Segment registers point to a place in memory where one of the following things begin:
 - Data storage
 - Code execution.

User-visible registers

- **Index registers** : The index register is employed for **indexed addressing** and its **initial value** is **0**. Generally, it is used for traversing the memory locations. After each reference, the index register is **incremented or decremented by 1**, depending upon the nature of the operation. Sometime the index register may be **auto indexed**.
- **Stack pointer register**: If there is user visible stack addressing, then typically the stack is in memory and there is a dedicated register that points to the top of the stack.
- **Condition Codes** (also referred to as flags) are bits set by the CPU hardware as the result of the operations. For example, an arithmetic operation may produce a positive, negative, zero or overflow result. In addition to the result itself being stored in a register or memory, a condition code is also set. The code may be subsequently be tested as part of a condition branch operation. Condition code bits are collected into one or more registers.

Control and Status Registers

- The control and status register holds the **address or data** that is important to **control the processor's operation**. The most important thing is that these registers are **not visible** to the users.
- Four registers are essential to instruction execution:



Control and Status Registers

- **Program Counter (PC):**The program counter is a processor register that holds the **address of the instruction that has to be executed next**. It is a processor which updates the program counter with the address of the next instruction to be fetched for execution.
- **Instruction Register (IR):** Contains the instruction most recently fetched. The fetched instruction is loaded into an IR, It helps in analyzing the opcode and operand present in the instruction.
- **Memory Address Register (MAR):**Contains the address of a location of main memory from where information has to be fetched or information has to be stored. Contents of MAR is directly connected to the address bus.
- **Memory Buffer Register (MBR):**Contains a word of data to be written to memory or the word most recently read. Contents of MBR is directly connected to the data bus.It is also known as Memory Data Register(MDR).
- The memory address registers (MAR) and memory buffer registers (MBR) are used to **move the data** between **processor** and **memory**.

Register Organization

- Apart from these specific register, we may have some temporary registers.
- **Processor Status Word**-All CPU designs include a register or set of registers, often known as the processor status word (PSW), that contains status information.
- The PSW typically contains condition codes plus other status information. Common fields or flags include the following:
 - **Sign**: Contains the sign bit of the result of the last arithmetic operation.
 - **Zero** :Set when the result is zero.
 - **Carry**: Set if an operation resulted in a carry (addition) into or borrow (subtraction) out of a high order bit.

Register Organization

- **Equal** : Set if a logical compare result is equal.
- **Overflow** : Used to indicate arithmetic overflow.
- **Interrupt enable/disable**: Used to enable or disable interrupts.
- **Supervisor**: Indicate whether the CPU is executing in supervisor or user mode.
- Certain privileged instructions can be executed only in supervisor mode, and certain areas of memory can be accessed only in supervisor mode
- **[note: The user state is the default (normal) state of operation, in which user programs are executed. The supervisor state is a special mode of operation to which the user has no access.** When it is in the supervisor state, the processor and its actions are entirely controlled by the Operating System (OS).
- The size of a register maybe **8, 16, 32, or 64 bits**. In 32-bit CPU, each register is 32 bits wide and it can manipulate 32 bits of data at a time. The modern PCs have 32-bit or 64-bit registers and are referred to as 32-bit processors and 64-bit processors.]

Addressing Mode and its Types

- **Addressing modes** specifies the way, the effective address of an operand is represented in the instruction.
- Generally, the programs are written in a high-level language, as it is a convenient way to define the variables and operations that the programmer needs to perform on the variables. Later, this program is compiled to generate the machine code. The machine code has low-level instructions.
- The low-level instruction has **opcode** and **operands**. Addressing mode has nothing to do with the opcode part. It focuses on presenting the operand's address in the instructions.

Types of Addressing Modes

- Register Addressing Mode
- Direct Addressing Mode
- Immediate Addressing Mode
- Register Indirect Addressing Mode
- Index Addressing Mode
- Auto Increment Mode
- Auto Decrement Mode
- Relative Addressing Mode

Before discussing the addressing modes, we must know about the term “effective address”.

Effective Address (EA):

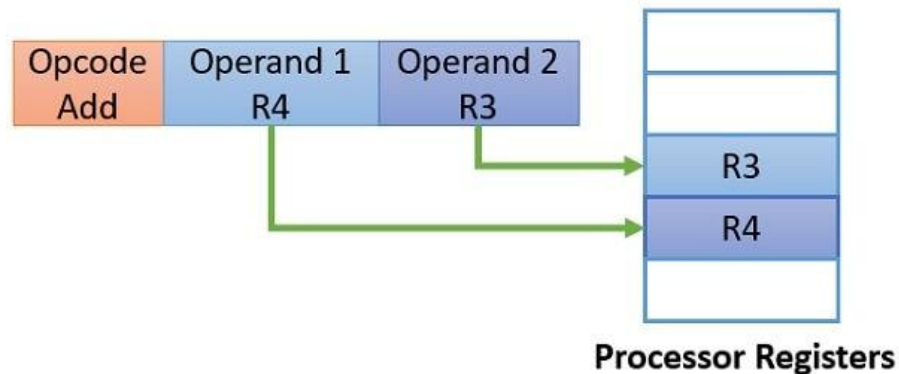
- Effective address is the **address of the exact memory location** where the **value of the operand** is present.

Types of Addressing Modes

- 1. Register Addressing Mode
- Every instruction includes operands; the operands can be a memory location, a processor register or an I/O device.
- The instruction which uses processor **registers** to represent operands is the instruction in **register addressing mode**.
- Here, the effective address is a register where the value of the operand is present.
- **EA=R**
- Below we have two instructions as our examples for register addressing mode.
- **Add R4, R3**
- **Load R3, R2**

Types of Addressing Modes

- 1. Register Addressing Mode
- In the examples above, the Add instruction uses registers to represent both of its operands. Similarly, the Load instruction also uses registers to represent both of its operands. So, the instruction above uses register addressing mode to describe the address of the operand. Below, we have a figure showing the Add instruction in the example above.



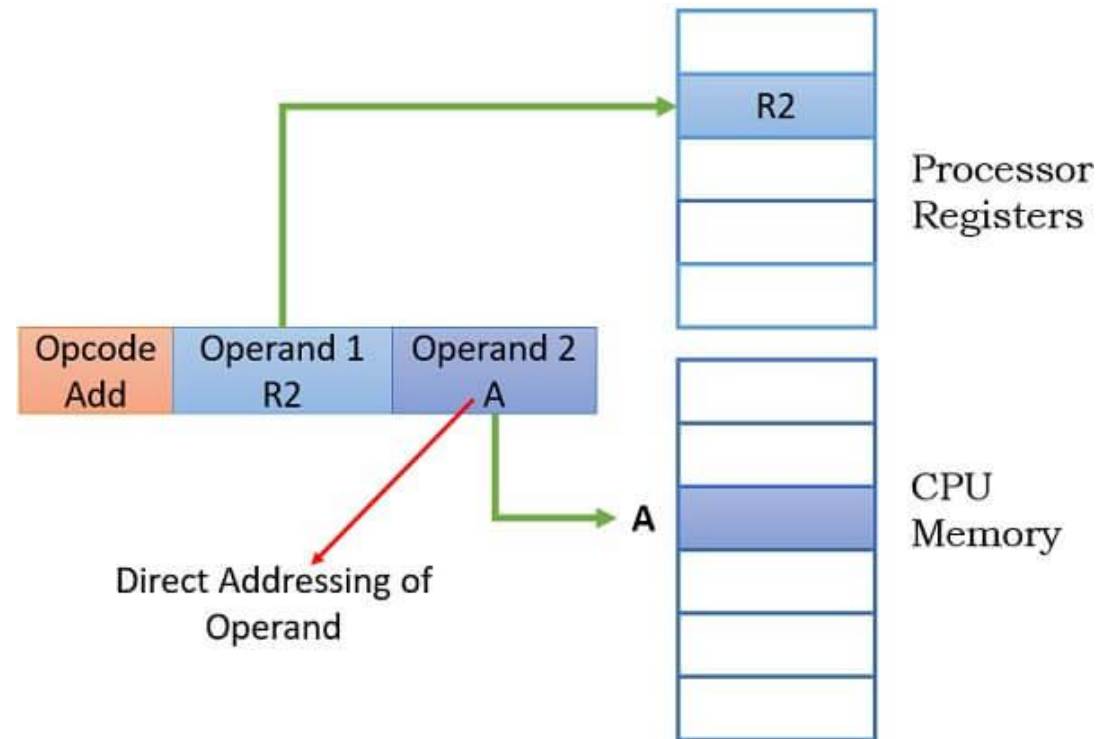
- **Advantage:** In the register addressing mode there are no memory references as the value to be operated is present in the register.
- **Disadvantage:** Registers have limited address space. So, it has a limit on the size of value that can be stored.

Types of Addressing Modes

- 2. Direct Addressing Mode
- The direct addressing mode is also known as **Absolute Addressing mode**. Here, the instruction contains the address of the **location in memory** where the value of the operand is stored.
- Here, the effective address is the address of memory location.
- **EA = A**
- For example, observe the examples below:
- **Add R2, A**
- **Store R2, B**
- The Add instruction includes the memory location A which has the value to be added to the content of register R2. Similarly, the Store instruction has the address of memory location B where the content of register R2 will be stored. Below we have a figure showing the direct addressing of the operand A in the Add instruction of the example above.

Types of Addressing Modes

- 2. Direct Addressing Mode



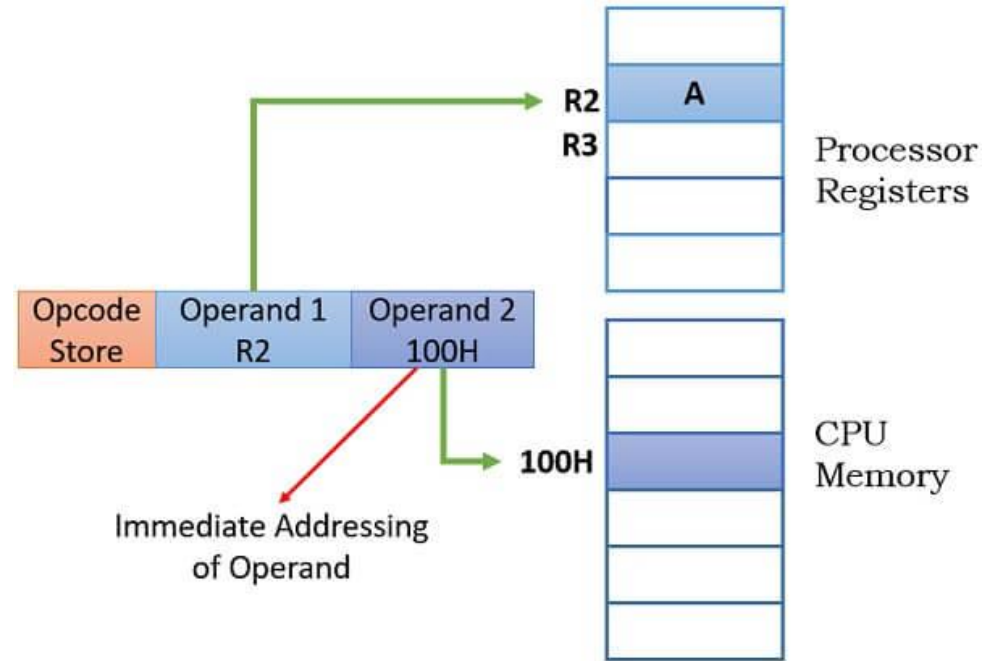
- Advantage:** Direct addressing mode is the simplest of all addressing mode.
- Disadvantage:** Direct addressing mode provides a limited address space.

Types of Addressing Modes

- 3. Immediate Addressing Mode
- In immediate addressing mode, the value of the operand is **explicitly mentioned** in the instruction. Here, effective address is not required as the operand is explicitly defined in instruction.
- Let us see the example of immediate addressing mode:
 - **Add R2, #100**
 - **Store R2, 100H**
- The Add instruction, adds 100 to R2's content . The # sign in front of the value indicates the immediate value to be operated. If a value does not have # sign in front of it then it is the address of a memory location.
- The next instruction Store considers the immediate value 100H as address as it does not have # sign in front of it. The Store instruction stores the content of R2 at memory location 100H. In the figure below we have shown the Store instruction of the above examples.

Types of Addressing Modes

- 3. Immediate Addressing Mode



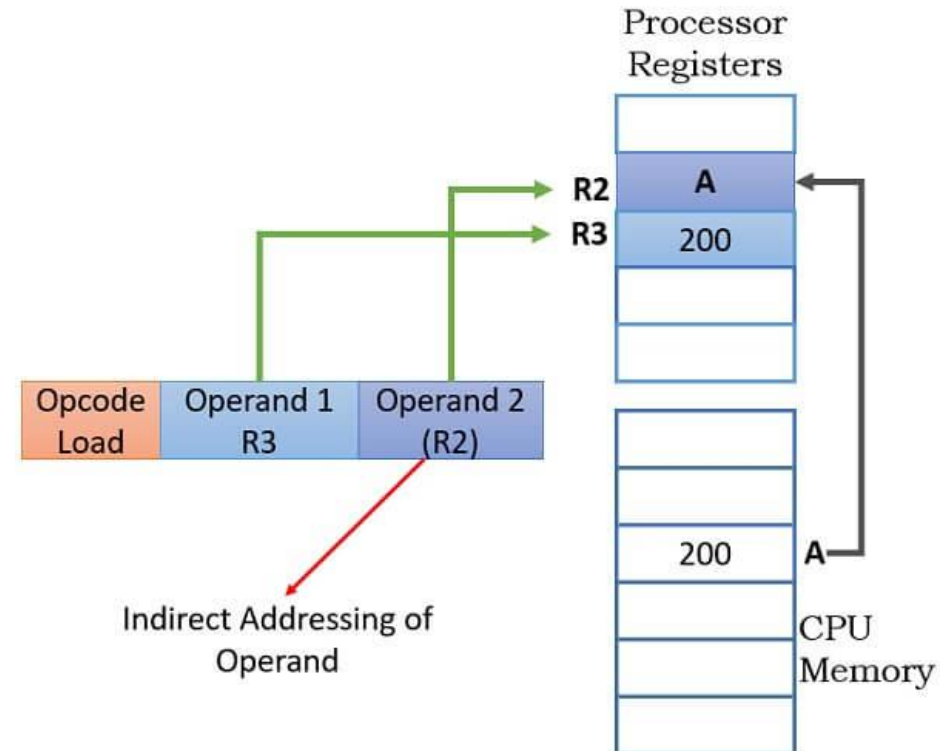
- **Advantage:** In the immediate addressing mode the memory reference is not required as the value is explicitly present in the instruction
- **Disadvantage:** The instruction format provides a limited size for the operand. So, the immediate addressing mode has limited space for immediate value.

Types of Addressing Modes

- 4. Register Indirect Addressing mode
- A **processor register** is used to hold the **address of a memory location** where the operand is placed. This addressing mode allows executing the same set of instructions for the different memory location. This can be done by incrementing the content of register thereby pointing the new location each time.
- In higher-level language, it is referred to as pointers. The indirect mode is denoted by placing the register inside the parenthesis.
- Here the effective address is the content of memory location present in the register.
EA=(R)
- Now, for example:
- **Load R3, (R2)** // Load R2, A
- The Load instruction above will load the value present at the memory location contained by register R2 in the register R3. The figure below shows how the register R3 gets loaded with the value stored in the memory location held by register R2.

Types of Addressing Modes

- 4. Register Indirect Addressing mode



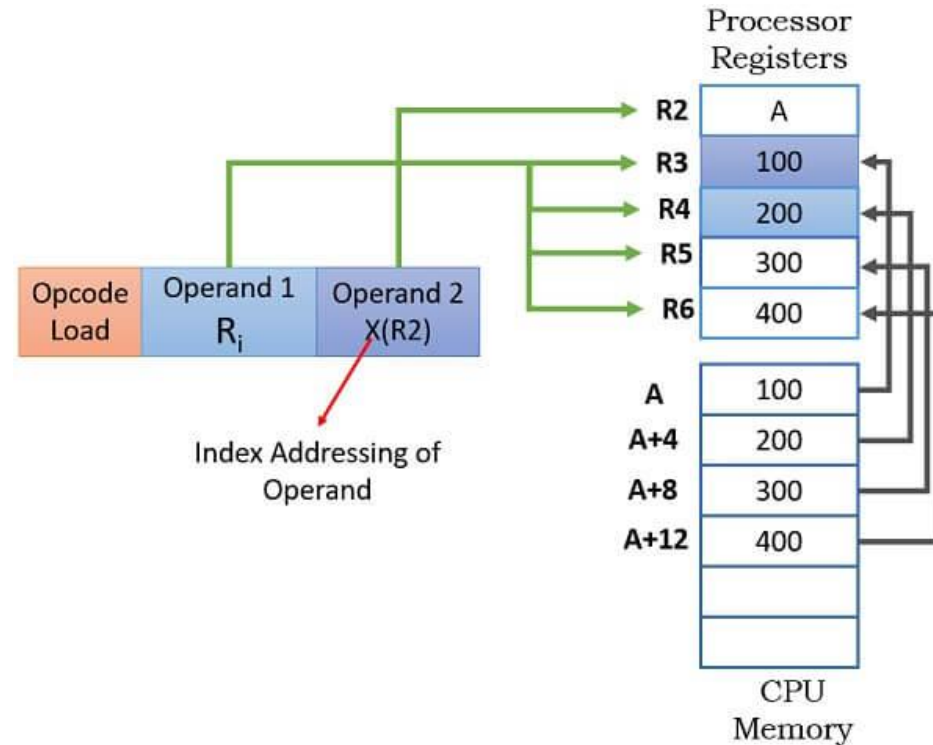
- **Advantage:** In the register indirect addressing mode the same set of instructions can be used multiple times.
- **Disadvantage:** In the register indirect addressing mode the number of memory reference is more.

Types of Addressing Modes

- 5. Index Addressing Mode
- Index addressing mode is helpful when the instructions in the program are accessing the **array or the large range of memory addresses**. In this mode, the effective address is generated by **adding a constant to the register's content**. The content of the register does not change.
- The symbolic representation of index addressing mode is denoted as:
- **X(R)**
- And the effective address is denoted by
- **$EA = X + (R)$**
- For example, consider the instruction below:
- **Load R2, A**
- **Load R3, (R2)**
- **Load R4, 4(R2)**
- **Load R5, 8(R2)**
- **Load R6, 12(R2)**
- The above instructions will load the register R3, R4, R5, R6 with the contents, present at the successive memory addresses from memory location A correspondingly.

Types of Addressing Modes

- 5. Index Addressing Mode
- Load R2, A
- Load R3, (R2)
- Load R4, 4(R2)
- Load R5, 8(R2)
- Load R6, 12(R2)



- **Advantage:** The index addressing mode provides flexibility to specify memory locations.
- **Disadvantage:** The index addressing mode is complex to implement.

Types of Addressing Modes

- 6. Auto Increment Addressing Mode
- In auto-increment addressing mode once the content of the register is **accessed** by the instruction the register's content is **incremented** to refer the next operand.
- Symbolically it is represented as below:
- **(R)+**
- Here, the effective address is content of the register as it is enclosed by parenthesis. The content of register which is referring to a memory location is incremented so that it could point the next memory location where the next operand is stored.

Types of Addressing Modes

- 7. Auto Decrement Addressing Mode
- It is just opposite of auto-increment mode. In auto decrement mode the content of the register is **decremented initially** and then the decremented content of the register is used as effective address.
- Symbolically it is presented as:
- **-(R)**
- The auto-increment and decrement mode help to implement the **stack structure**.

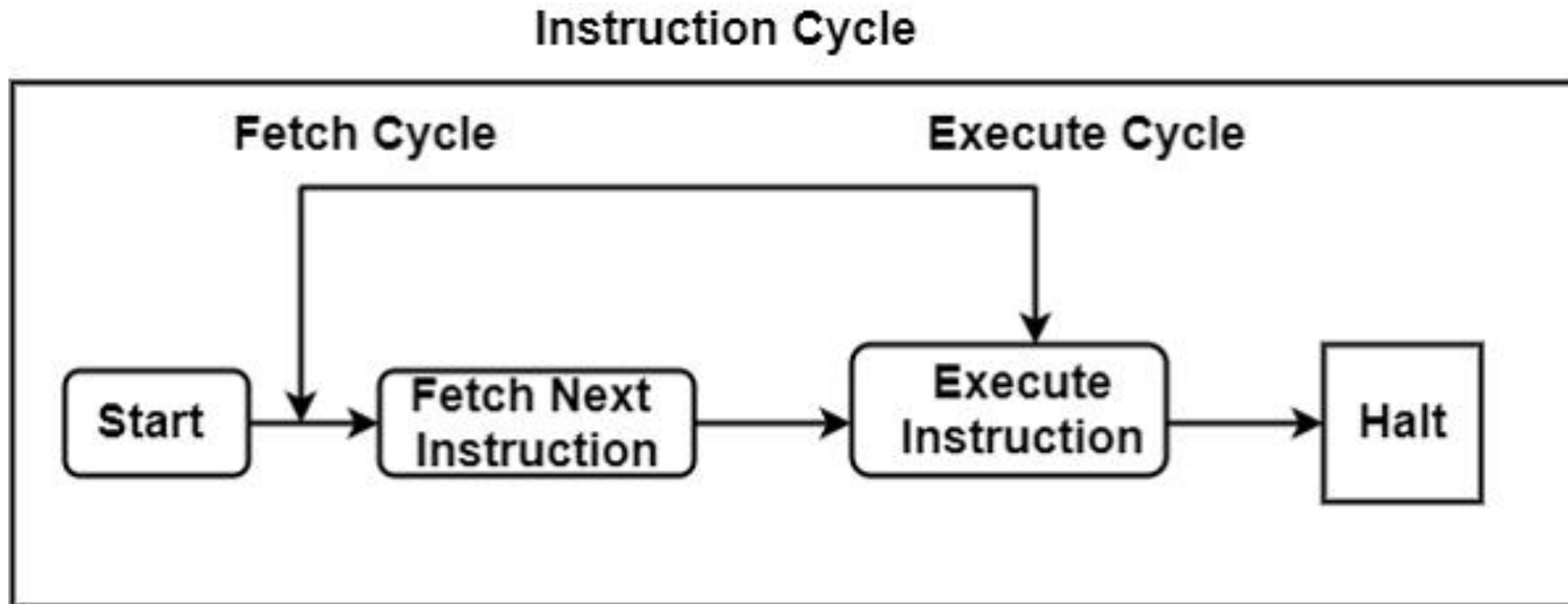
Types of Addressing Modes

- 8. Relative Addressing Mode
- In the content above we have discussed the **index addressing mode**. There we were adding a constant to the register content to refer the next operand address. In some computer instead of a register, the **program counter** is used.
- The symbolic representation of relative address mode is
- **$X(PC)$**
- The effective address for it would be:
- **$EA = X + (PC)$**
- As here the operand addresses are found relative to the program counter. That's why it is referred to as relative address mode.
- **Advantage:** Relative addressing mode doesn't require memory references.
- **Disadvantage:** Relative addressing mode doesn't have any disadvantage as such.

Instruction Cycle

- A program consisting of the memory unit of the computer includes a series of instructions. The program is implemented on the computer by going through a cycle for each instruction.
- In the basic computer, each instruction cycle includes the following procedures –
- **It can fetch instruction from memory.**
- **It is used to decode the instruction.**
- **It can read the effective address from memory if the instruction has an indirect address.**
- **It can execute the instruction.**
- After the following four procedures are done, the control switches back to the first step and repeats the similar process for the next instruction.

Instruction Cycle



Instruction Cycle

- **Fetch Cycle**

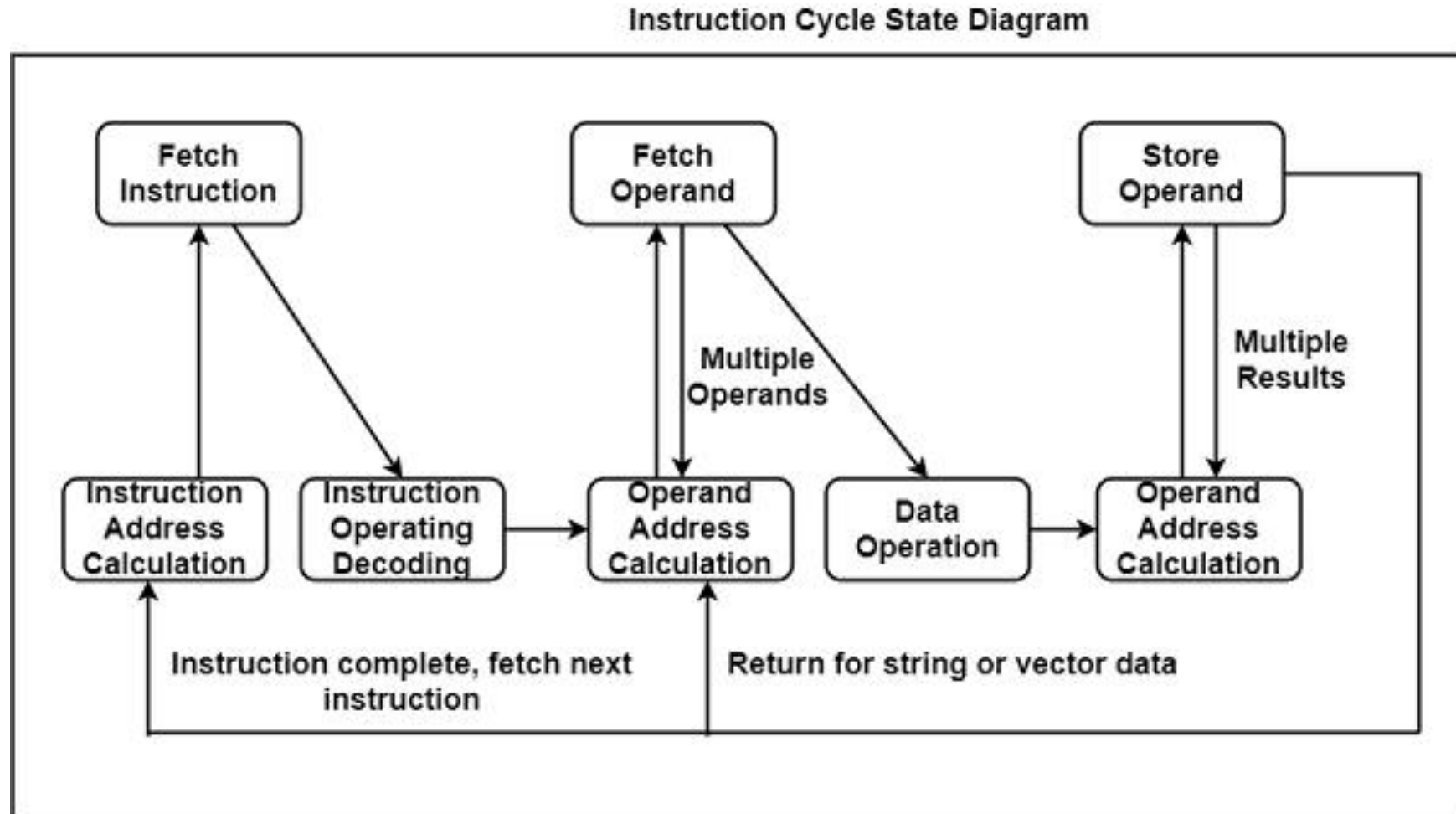
- The address instruction to be implemented is held at the program counter. The processor fetches the instruction from the memory that is pointed by the PC.
- Next, the PC is incremented to display the address of the next instruction. This instruction is loaded onto the instruction register. The processor reads the instruction and executes the important procedures.

Instruction Cycle

- **Execute Cycle**

- The data transfer for implementation takes place in two methods are as follows –
- **Processor-memory** – The data sent from the processor to memory or from memory to processor.
- **Processor-Input/Output** – The data can be transferred to or from a peripheral device by the transfer between a processor and an I/O device.
- In the execute cycle, the processor implements the important operations on the information, and consistently the control calls for the modification in the sequence of data implementation.
- These two methods associate and complete the execute cycle.

State Diagram for Instruction Cycle



State Diagram for Instruction Cycle

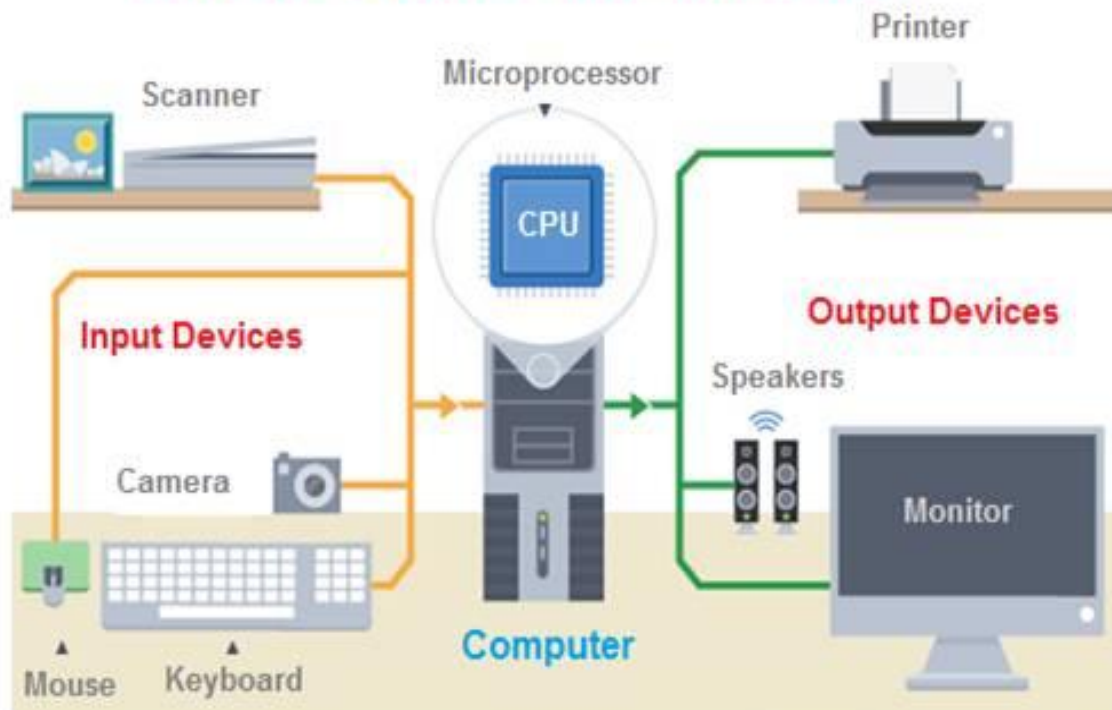
- **Instruction Address Calculation** – The address of the next instruction is computed. A permanent number is inserted to the address of the earlier instruction.
- **Instruction Fetch** – The instruction is read from its specific memory location to the processor.
- **Instruction Operation Decoding** – The instruction is interpreted and the type of operation to be implemented and the operand(s) to be used are decided.
- **Operand Address Calculation** – The address of the operand is evaluated if it has a reference to an operand in memory or is applicable through the Input/Output.
- **Operand Fetch** – The operand is read from the memory or the I/O.
- **Data Operation** – The actual operation that the instruction contains is executed.
- **Store Operands** – It can store the result acquired in the memory or transfer it to the I/O.

Computer Bus

- The computer system consist of number of internal and external components . These components are physically interconnected and communicate with each other through a network of wires running across the computer system.
- These wires are referred as computer buses . The buses are essential to the functioning of the computer system.
- The computer buses can be in the form of wired cables or electrical wires embedded in the computer motherboard PCB (Printed Circuit Board) visible on the rear side of motherboard .
- It is important for computer science professional to study the computer system bus architecture , technical features of these buses such as bus width and bus speed and its overall impact on the system performance.

Computer Bus

Computer System - External Devices



Computer - Internal Components



Computer Bus

- A bus is a common communication pathway used in a computer system through which information flows from one computer component to another.
- The computer bus system is a network of buses which physically connect all the components with wires (actual bus wires OR circuit wires on the motherboard) .
- The bus system consist of different types of buses depending upon the components being connected and the function assigned to the bus .
- A bus can consist of set of wires grouped together as connection wire or a printed circuit boards which carry the data and other commands (instructions) from the CPU to the memory and to various other components connected to the system.
- The bus performance is an important parameter to access the computer system performance . The bus width and the bus speed affects the system performance

Computer Bus

Buses - Connection Cables



Buses - Motherboard Buses



Computer Bus

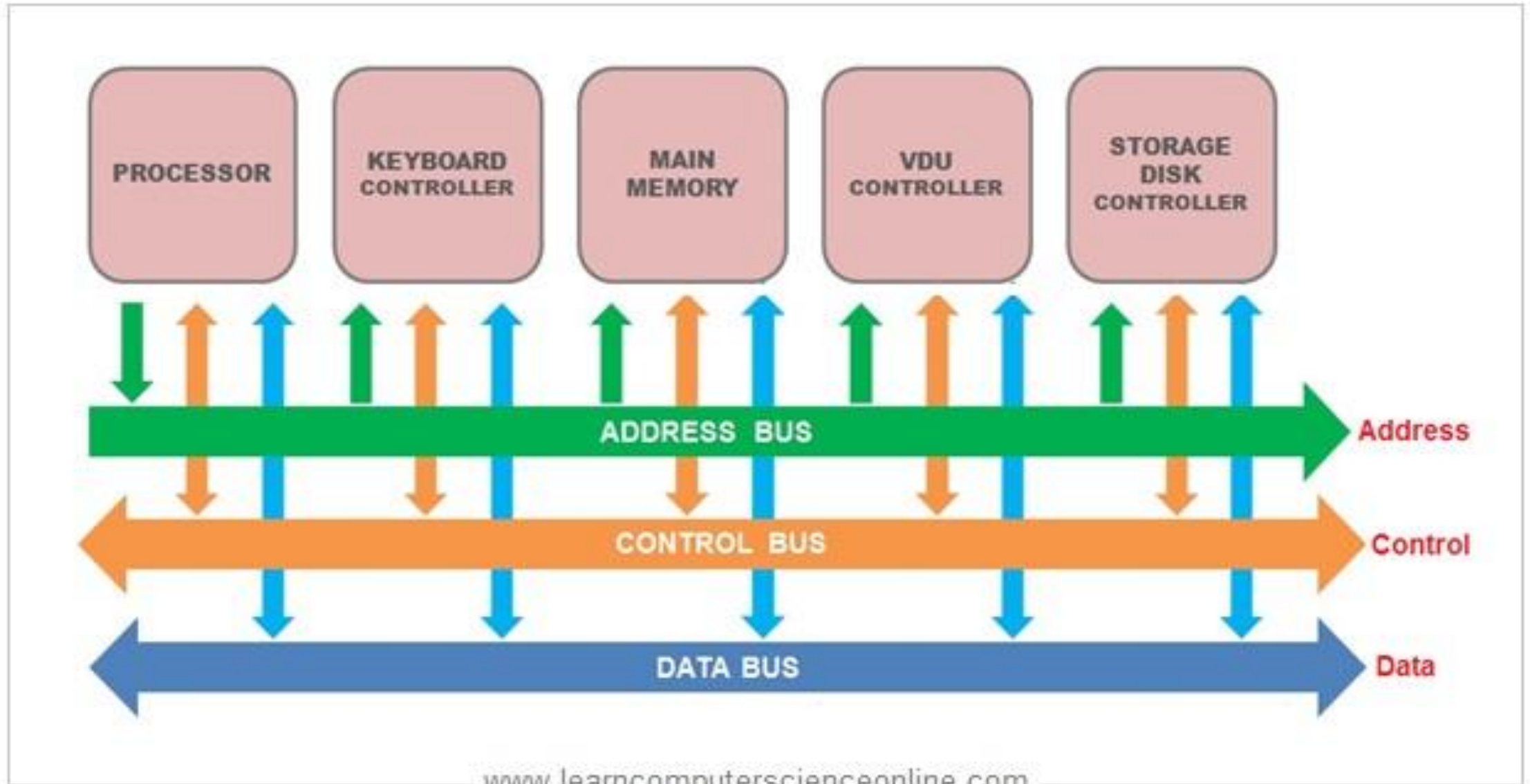
- **Types Of Computer Buses**

1. Data Bus , 2. Address Bus , 3. Control Bus.

- The data bus is a bidirectional bus and can carry the data in both the direction along the data bus. For example , the CPU can send the data to be stored into the RAM .
- Similarly, the CPU can also perform the fetch operation for retrieving the data from the specific memory location.

Computer Bus

Motherboard Bus Structure



Data Bus

- In computer architecture , the data bus is a wired connection dedicated for the transmitting the data between the CPU , peripheral devices and other hardware components . The data bus is a part of the system bus in addition to address bus and the control bus.
- A data bus has many different features , but one of the most important feature is the bus width. The width of a data bus refers to the number of bits that the bus can carry at a time.
- For example , a 16 Bit wide data bus can carry 16 bits of data simultaneously between the CPU and the system component such as main memory RAM (Random Access Memory).
- The Common data bus widths include 8 bit , 16 bit , 32 bit and 64 bit . The wider the bus width , faster would be the data flow on the data bus and thus better system performance.

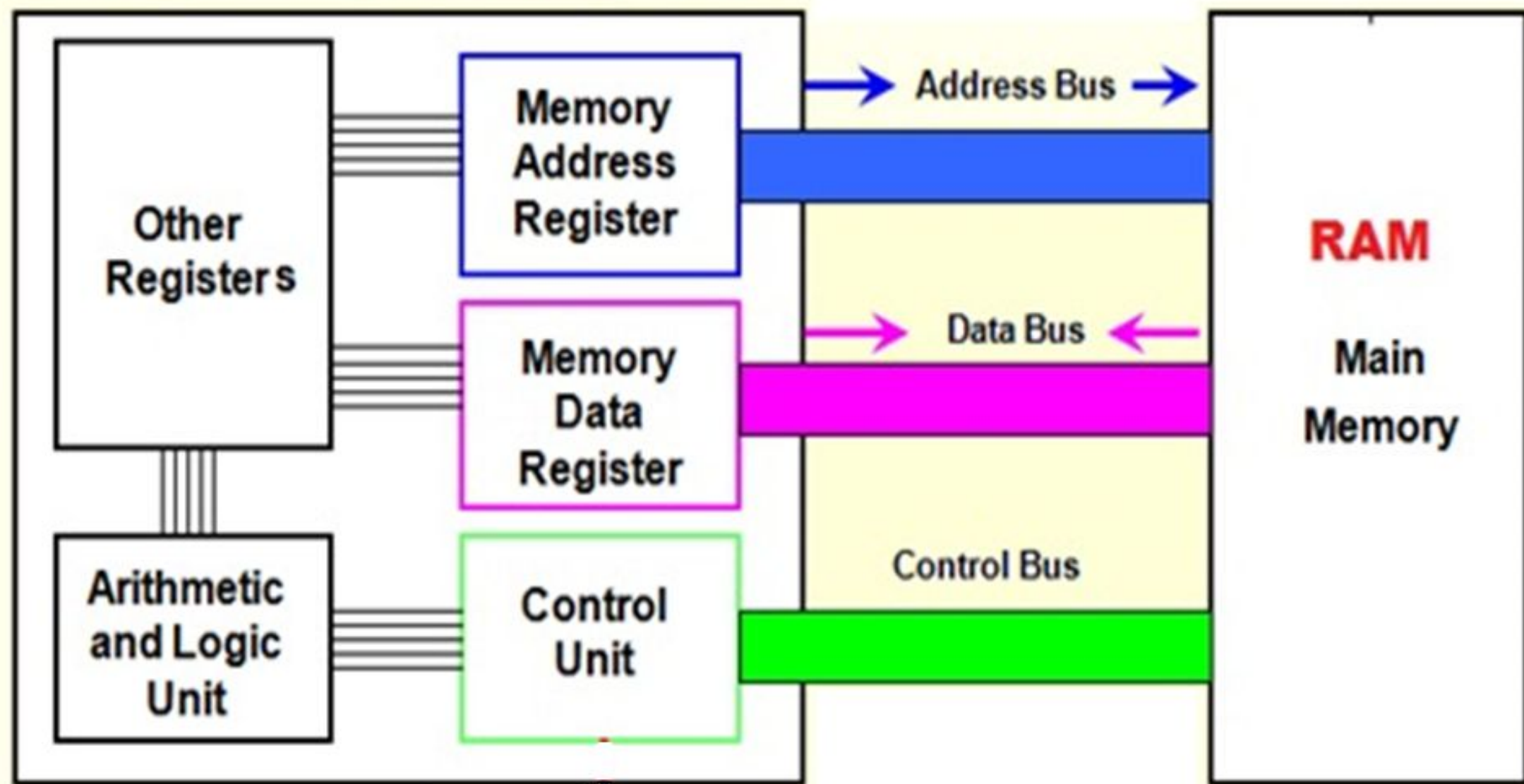
Control Bus

- The CPU (Microprocessor) contains a control unit which controls the functioning of all other components connected to the computer system. The control bus is used to transfer the control signals from one component to another component .
- A control bus is a computer bus that is used by the CPU to communicate with the devices that are connected to the computer system. These devices are connected with the help of cables and printed circuits board such as motherboard.
- The Control Bus is a part of System Bus in addition to Data Bus and Address Bus.
- The Central Processing Unit (CPU) transmits different types of control signals to the system components. The devices also communicate with CPU by transmitting the control signals using the control bus.
- The control bus is a bidirectional and assists the CPU in synchronizing control signals to the internal components and the external devices connected to the system.
- The control bus transmits the control signals such as device interrupt signal , memory read or write signals and status signals.

Address Bus

- The computer program consist of number of program instructions. These instructions direct the CPU to perform desired operation.
- Address bus is **unidirectional** because data flow in one direction, from microprocessor to memory or from microprocessor to Input/output devices
- The operating system loads the program instructions and the data into the main memory . The CPU executes the program instructions one-by-one by fetching the program instructions from the main memory RAM (Random Access Memory) .
- In order to perform the memory read or write operation from the main memory RAM , the CPU sends either read or write control signal on the control bus and address of the memory location along the “Address Bus” from where the operation is to be performed .
- The address bus is a part of the “System Bus” along with the data bus and the control bus which we have discussed .

CPU - Central Processing Unit



Input / Output Organization

- The computer system's input/output (I/O) architecture is its interface to the outside world.
- Each I/O module interfaces to the system bus and controls one or more peripheral devices.
- There are several reasons why an I/O device or peripheral device is not directly connected to the system bus. Some of them are as follows –
- There are a wide variety of peripherals with various methods of operation. It would be impractical to include the necessary logic within the processor to control several devices.
- The data transfer rate of peripherals is often much slower than that of the memory or processor. Thus, it is impractical to use the high-speed system bus to communicate directly with a peripheral.
- Peripherals often use different data formats and word lengths than the computer to which they are attached.
Thus, an I/O module is required.

Input / Output Modules

- The major functions of an I/O module are categorized as follows –
- Control and timing
- Processor Communication
- Device Communication
- Data Buffering
- Error Detection
- During any period of time, the processor may communicate with one or more external devices in unpredictable manner, depending on the program's need for I/O.
- The internal resources, such as main memory and the system bus, must be shared among a number of activities, including data I/O.

Control & timings:

- The I/O function includes a control and timing requirement to co-ordinate the flow of traffic between internal resources and external devices.
- **For example, the control of the transfer of data from an external device to the processor might involve the following sequence of steps –**
- The processor interacts with the I/O module to check the status of the attached device.
- The I/O module returns the device status.
- If the device is operational and ready to transmit, the processor requests the transfer of data, by means of a command to the I/O module.
- The I/O module obtains a unit of data from external device.
- The data are transferred from the I/O module to the processor.
- If the system employs a bus, then each of the interactions between the processor and the I/O module involves one or more bus arbitrations.

Processor & Device Communication

- During the I/O operation, the I/O module must communicate with the processor and with the external device.
- **Processor communication involves the following –**
- **Command decoding :**
 - The I/O module accepts command from the processor, typically sent as signals on control bus.
- **Data :**
 - Data are exchanged between the processor and the I/O module over the data bus.
- **Status Reporting :**
 - Because peripherals are so slow, it is important to know the status of the I/O module. For example, if an I/O module is asked to send data to the processor(read), it may not be ready to do so because it is still working on the previous I/O command. This fact can be reported with a status signal. Common status signals are BUSY and READY.
- **Address Recognition :**
 - Just as each word of memory has an address, so thus each of the I/O devices. Thus an I/O module must recognize one unique address for each peripheral it controls.
 - On the other hand, the I/O must be able to perform device communication. This communication involves command, status information and data.

Data Buffering and Error Detection:

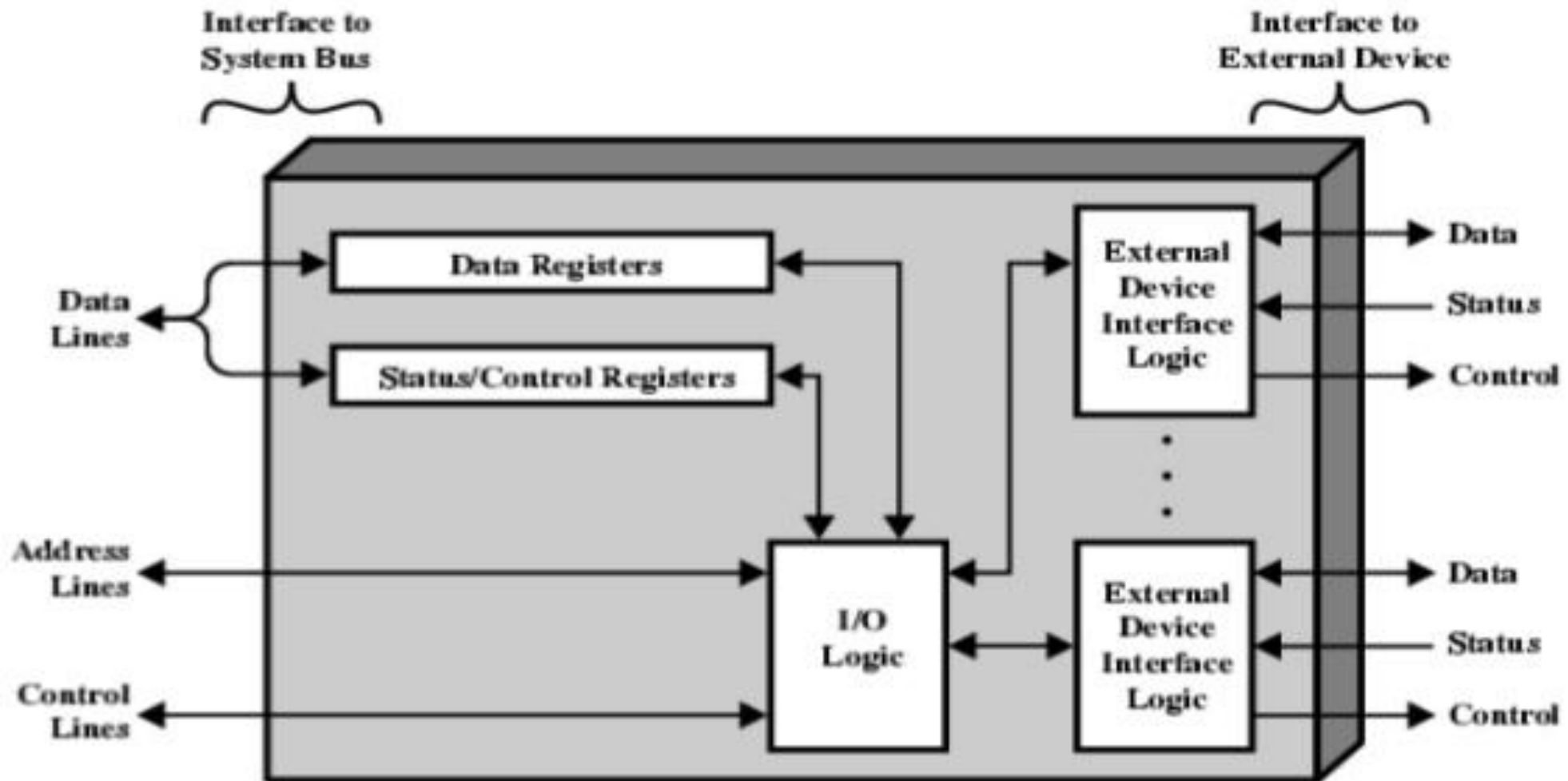
- **Data Buffering:**

- An essential task of an I/O module is data buffering. The data buffering is required due to the mismatch of the speed of CPU, memory and other peripheral devices. In general, the speed of CPU is higher than the speed of the other peripheral devices. So, the I/O modules store the data in a data buffer and regulate the transfer of data as per the speed of the devices.
- In the opposite direction, data are buffered so as not to tie up the memory in a slow transfer operation. Thus the I/O module must be able to operate at both device and memory speed.

- **Error Detection:**

- Another task of I/O module is error detection and for subsequently reporting error to the processor. One class of error includes mechanical and electrical malfunctions reported by the device (e.g. paper jam).
- Another class consists of unintentional changes to the bit pattern as it is transmitted from devices to the I/O module.

I/O MODULE STRUCTURE DIAGRAM



I/O Module

- There will be many I/O devices connected through I/O modules to the system. Each device will be identified by a unique address.
- When the processor issues an I/O command, the command contains the address of the device that is used by the command. The I/O module must interpret the address lines to check if the command is for itself.
- Generally in most of the processors, the processor, main memory and I/O share a common bus(data address and control bus).

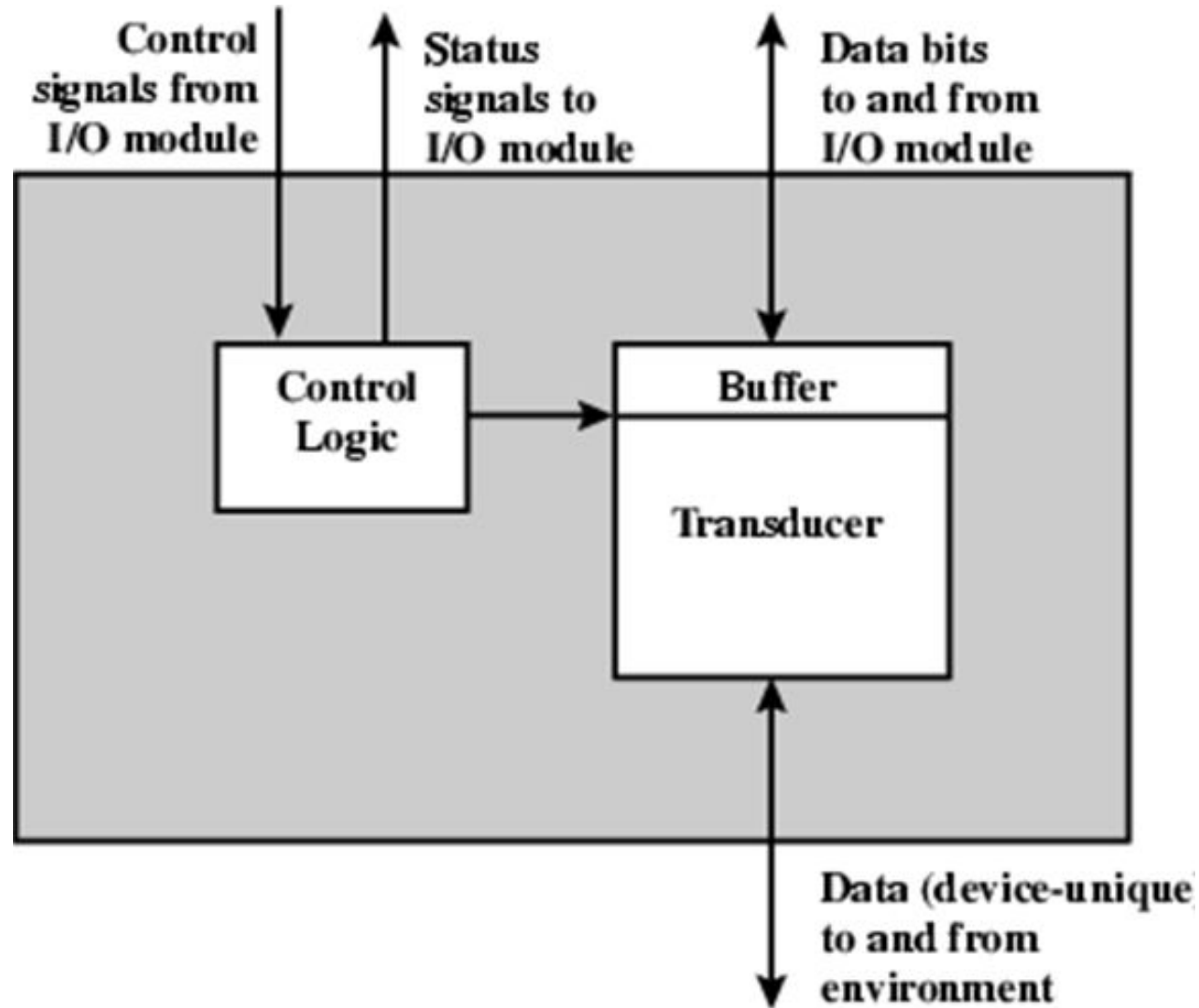
External Devices

- I/O operations are accomplished through external devices that provide a means of exchanging data between external environment and computer.
- An external device attaches to the computer by a link to an I/O module. An external device linked to an I/O module is called peripheral device or peripheral.
- External Devices can be categorized as:
 - Human readable: suitable for communicating with computer user. For example - video display terminals and printers.
 - Machine readable: suitable for communicating with equipment. For example – disks, sensor, actuators used in robotics application.
 - Communication: suitable for communicating with remote devices. For example – modems.

External Devices

- External devices are not connected directly to the system bus because they have a wide range of control logics, as well as data transfer speeds and formats.
- Virtually all external devices have buffers, control signals, status signals, and data bits.

Block Diagram of an External Device



External Devices

- Data - bits sent to or received from the I/O module
- Control signals - determine the function that the device will perform such as send data to the I/O module, accept data from the I/O module, report status, or perform some control function particular to the device
- Status signals - indicate the state of the device e.g. READY/NOT-READY to show whether the device is ready for data transfer
- Control logic - interprets commands from the I/O module to operate the device
- Transducer - converts data from electrical signals to other forms of energy during output and from other forms to electrical during input
- Buffer - temporarily holds data being transferred between I/O module and the external device

I/O Module Addressing

- I/O stands for input/output. **I/O is a term to describe communication between the outer world, including humans and a computer using peripheral devices.**
- Some of the most common peripheral devices attached to a computer are keyboards, mouse, monitors, network adapters, and printers. Peripheral devices exchange information between the outer world and computer CPUs. Some I/O devices act as input devices, some as output devices, while others act as both.
- We pass the information to I/O devices from the CPU. We store the information passed from I/O devices to the CPU in memory. The CPU processes the data stored in memory and helps in transferring the data in an I/O operation.
- **In order to work efficiently within a system driven by a processor, I/O devices need an interface.** It's popularly known as the I/O interface. In general, the communication between the CPU and I/O devices occurs via a bus.
- The role of an I/O interface is to identify the address of the devices generated by the CPU. Additionally, an I/O interface includes facilitates an interface for the CPU to communicate with I/O devices.

Two types of addressing are possible –

- Memory-mapped I/O
- Isolated or I/O mapped I/O

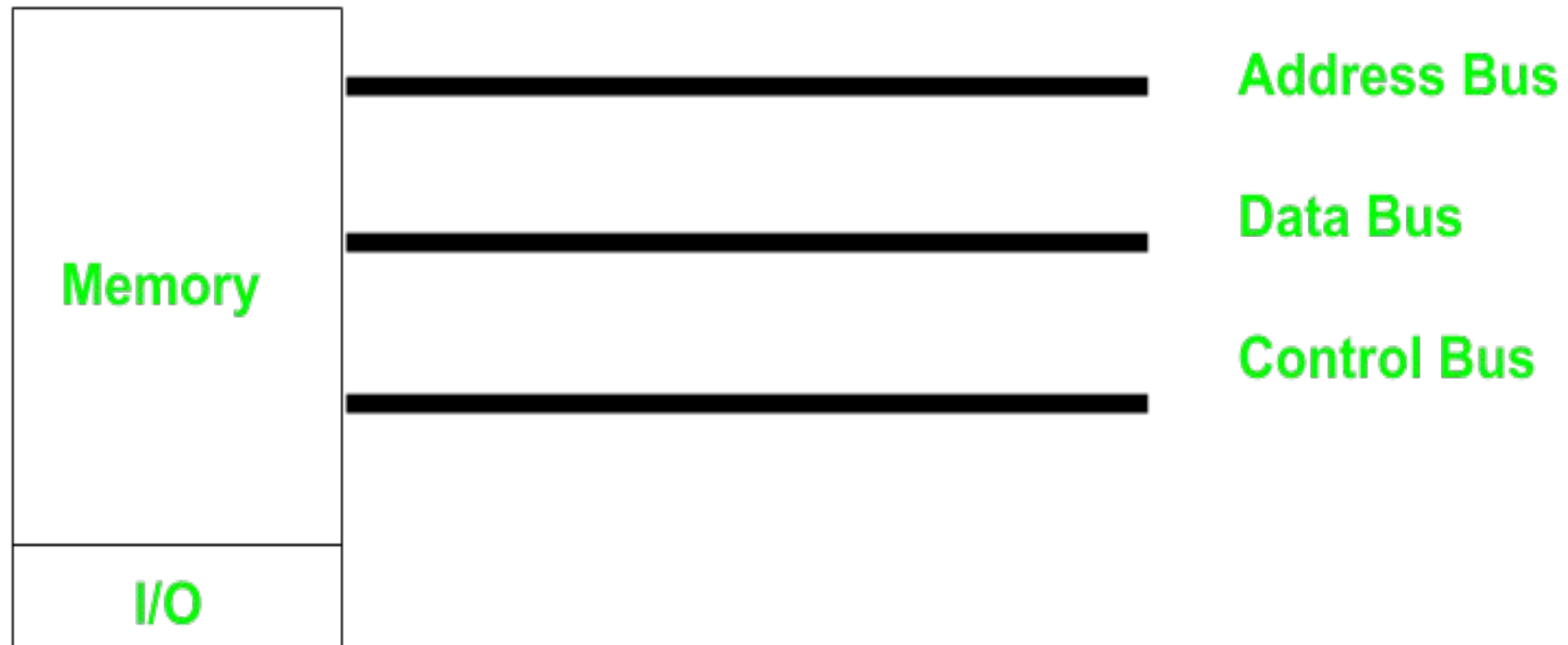
Memory-mapped I/O

- There're three types of buses required for I/O communication: address bus, data bus, and control bus. We assign an address to each I/O device for the CPU to communicate to that device using its address.
- **In memory-mapped I/O, both memory and I/O devices use the same address space.** We assign some of the memory addresses to I/O devices. The CPU treats I/O devices like computer memory. The CPU either communicates with computer memory or some I/O devices depending on the address. Therefore, we reserve a part of the address space for I/O devices, which is not available for computer memory.

Memory-mapped I/O

- **In the case of memory-mapped I/O, all the buses are the same for both memory and I/O devices.**
- Therefore, building a CPU that uses memory-mapped I/O is easier and cheaper. Additionally, such CPUs consume less power due to reduced complexity. One advantage of memory-mapped I/O is that we don't need separate instruction sets for accessing I/O devices. Instructions used for accessing memory can be easily used for accessing I/O devices.

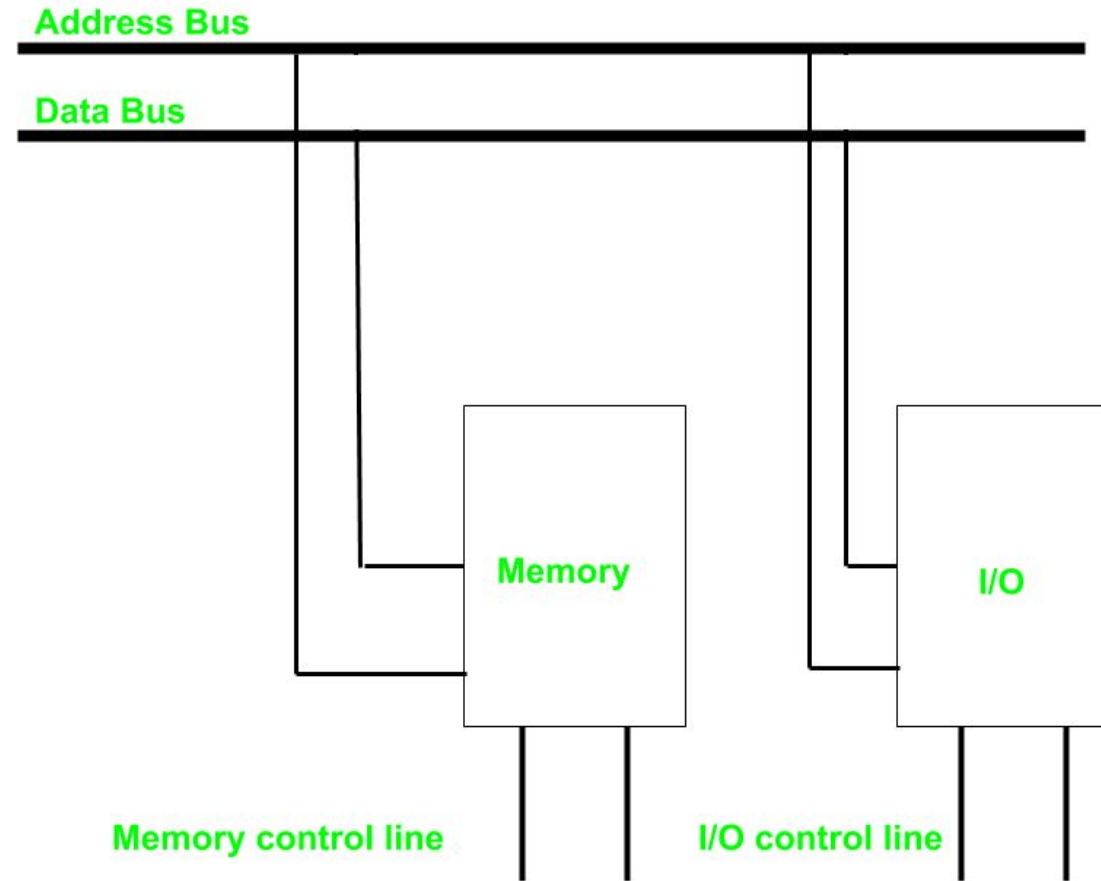
Memory-mapped I/O



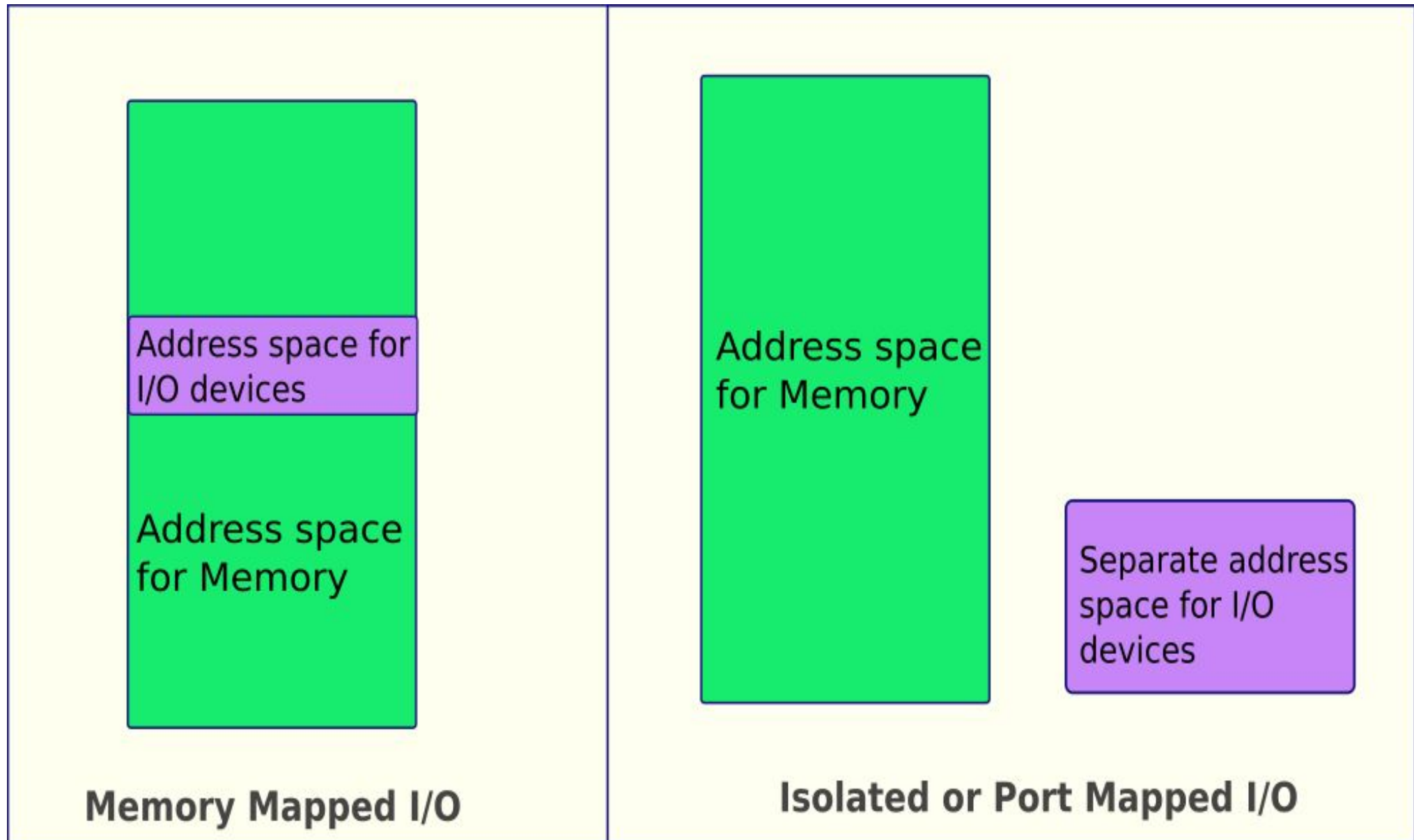
Isolated or I/O mapped I/O

- In the case of isolated I/O, we provide a separate address space other than a memory address space to I/O devices. The addresses of I/O devices are also referred to as ports. I/O devices and memory use the same address and data bus. However, the control bus is different for data and memory:
- Therefore, isolated I/O becomes costlier compared to memory-mapped I/O. The isolated I/O technique has its own dedicated instruction set for accessing I/O devices. The CPUs that use isolated I/O are bigger and more complex to build.
- For I/O devices, it doesn't matter whether memory mapped or isolated I/O is implemented. I/O devices behave in the same manner independent of the I/O method implemented internally.

Isolated or I/O mapped I/O



Isolated I/O	Memory-mapped I/O
Different address spaces are used for computer memory and I/O devices. I/O devices have dedicated address space.	Same address space is used for memory and I/O devices.
Separate control unit and control instructions are used in case of I/O devices.	Control units and instructions are same for memory and I/O devices.
More complex and costlier than memory-mapped I/O as more bus are used.	Easier to build and cheap as it's less complex.
Entire address space can be used by memory as I/O devices have separate address space.	Some part of the address space of computer memory is consumed by I/O devices as address space is shared.
Computer memory and I/O devices use different control instructions for read write.	Computer memory and I/O devices can both use same set of read and write instructions.
Separate control bus is used for computer memory and I/O devices. Though same address and data bus are used.	Address, data and control bus are same for memory and I/O devices.



DMA

- Direct memory access (DMA) is a **mode of data transfer** between the memory and I/O devices. This happens **without the involvement** of the processor.
- We have two other methods of data transfer, **programmed I/O** and **Interrupt driven I/O**. Let's revise each and get acknowledge with their drawbacks.
- In **programmed I/O**, the processor keeps on scanning whether any device is ready for data transfer. If an I/O device is ready, the processor **fully dedicates** itself in transferring the data between I/O and memory. It transfers data at a **high rate, but it can't get involved in any other activity** during data transfer. This is the major **drawback** of programmed I/O.

DMA

- In **Interrupt driven I/O**, whenever the device is ready for data transfer, then it raises an **interrupt to processor**.
- Processor completes executing its ongoing instruction and saves its current state.
- It then switches to data transfer which causes a **delay**.
- Here, the processor doesn't keep scanning for peripherals ready for data transfer.
- But, it is **fully involved** in the data transfer process. So, it is also not an effective way of data transfer.

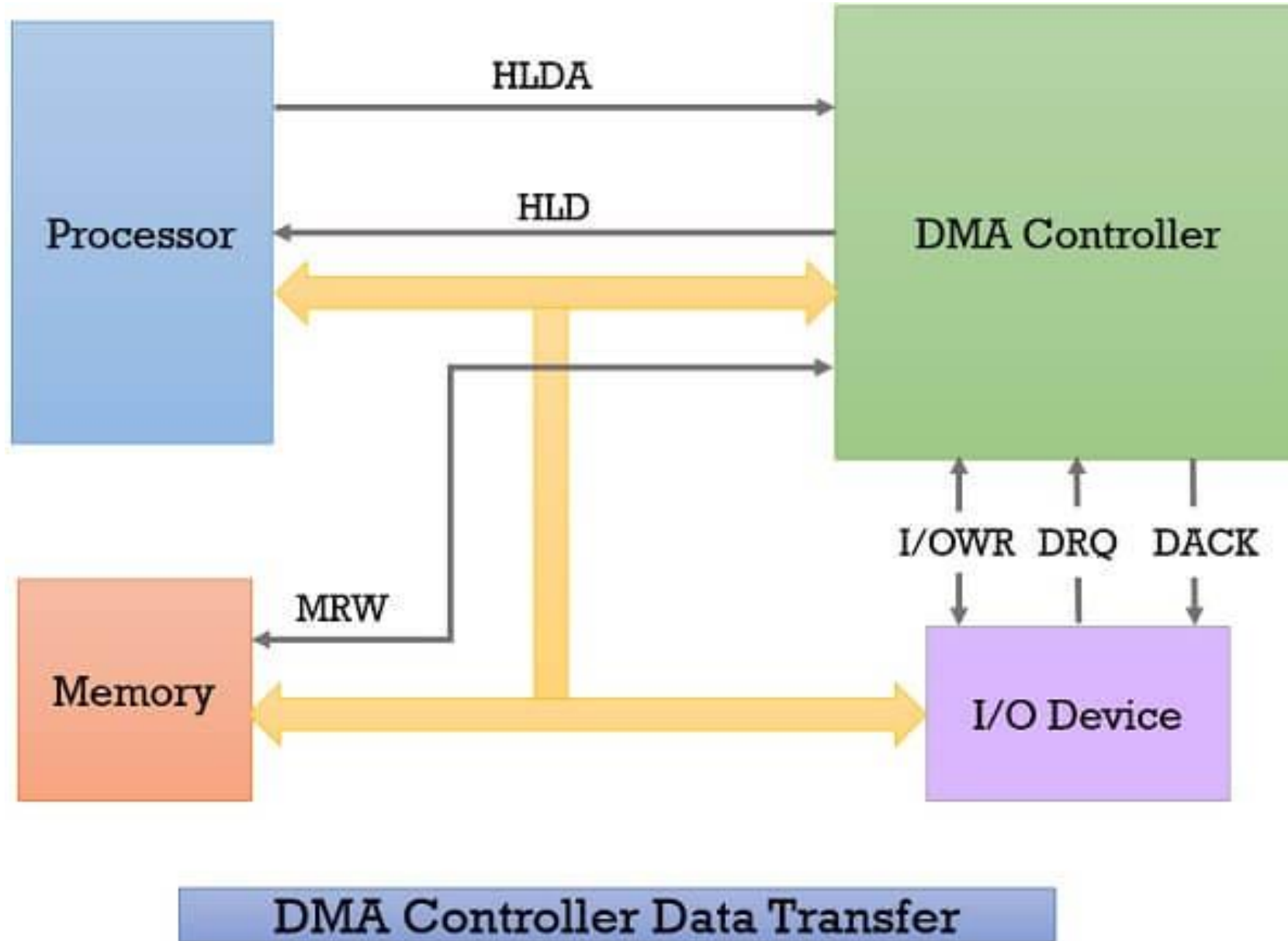
DMA

- The above two modes of data transfer are not useful for transferring a large block of data. But, the DMA controller completes this task at a faster rate and is also effective for transfer of large data block.
- The DMA controller transfers the data in three modes:
- **Burst Mode:** Here, once the DMA controller gains the charge of the system bus, then it releases the system bus only after **completion** of data transfer. Till then the CPU has to wait for the system buses.
- **Cycle Stealing Mode:** In this mode, the DMA controller **forces** the CPU to stop its operation and **relinquish the control over the bus** for a **short term** to DMA controller. After the **transfer of every byte**, the DMA controller **releases** the **bus** and then again requests for the system bus. In this way, the DMA controller steals the clock cycle for transferring every byte.
- **Transparent Mode:** Here, the DMA controller takes the charge of system bus only if the **processor does not require the system bus**.

Direct Memory Access Controller & it's Working

- DMA controller is a **hardware unit** that allows I/O devices to access memory directly without the participation of the processor. Here, we will discuss the working of the DMA controller. Below we have the diagram of DMA controller that explains its working:

DMA Block Diagram



Direct Memory Access Controller & it's Working

1. Whenever an I/O device wants to transfer the data to or from memory, it sends the DMA request (**DRQ**) to the DMA controller. DMA controller accepts this DRQ and asks the CPU to hold for a few clock cycles by sending it the Hold request (**HLD**).
2. CPU receives the Hold request (HLD) from DMA controller and relinquishes (hand over) the bus and sends the Hold acknowledgement (**HLDA**) to DMA controller.
3. After receiving the Hold acknowledgement (HLDA), DMA controller acknowledges I/O device (**DACK**) that the data transfer can be performed and DMA controller takes the charge of the system bus and transfers the data to or from memory.
4. When the data transfer is accomplished, the DMA raise an **interrupt** to let know the processor that the task of data transfer is finished and the processor can take control over the bus again and start processing where it has left.
 - Now the DMA controller can be a separate unit that is shared by various I/O devices, or it can also be a part of the I/O device interface.

Why DMA?

- Transferring the data without the involvement of the processor will **speed up** the read-write task.
- DMA **reduces the clock cycle** requires to read or write a block of data.
- Implementing DMA also **reduces the overhead** of the processor.